
Pre-Requisite Verification Document – PHP MVC Project (XAMPP Edition)

1. Git – Understanding & Installation

Verify Git is installed

```
git --version
```

Expected: `git version 2.x.x` or higher.

If not installed, download from git-scm.com.

Basic Git commands (self-test – same as before)

- How do you clone a repository?
- How do you stage a file?
- How do you commit with a message?
- How do you check the status?
- How do you view commit history?

GitHub account & push

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"

# Test HTTPS connection (easier on Windows)
git ls-remote https://github.com/yourusername/yourrepo.git
```

Repository setup

- ☐ Created a repository on GitHub.
- ☐ Cloned it locally (e.g., `git clone https://...`).
- ☐ Created a `.gitignore` with:

```
/vendor/
.env
config/db_config.php
/public/uploads/
*.log
```

2. PHP Environment (XAMPP)

XAMPP installs PHP in `C:\xampp\php`.

Make sure that folder is in your **System PATH** so you can run `php` from any command prompt.

Verify PHP version

```
php -v
```

Expected: PHP 7.4.x or 8.x.

Verify required extensions are enabled

Look inside `C:\xampp\php\php.ini` and ensure these lines are **not commented** (remove leading `;`):

```
extension=mysqli
extension=pdo_mysql
extension=fileinfo
extension=session      ; usually enabled by default
extension=json          ; usually enabled
```

Or run this in command prompt (rough check):

```
php -m | findstr /i "pdo_mysql mysqli session fileinfo json"
```

Verify you can run PHP built-in server

```
cd C:\xampp\htdocs\your-project-folder
php -S localhost:8000 -t public/
```

Then open `http://localhost:8000` in your browser.

3. MySQL / MariaDB (via XAMPP)

Verify MySQL is running

- Open **XAMPP Control Panel**.
- Check that **MySQL** is started (green “Running”).
- If not, click “Start”.

Connect and create a test database

```
mysql -u root -p
```

(Default XAMPP root password is **empty** – just press Enter.)

Inside MySQL:

```
CREATE DATABASE IF NOT EXISTS test_mvc;  
SHOW DATABASES;  
EXIT;
```

User privileges

Default XAMPP `root` user has full privileges.

For production, create a separate user, but for development `root` (empty password) is fine.

4. Composer (Highly Recommended)

Verify Composer is installed

```
composer --version
```

If not installed: download [Composer for Windows](#) – it will automatically detect your XAMPP PHP.

Basic commands

- `composer install`
 - `composer dump-autoload`
-

5. Project Folder Structure (Create inside `C:\xampp\htdocs\`)

```
your-project-root/  
├── config/  
│   └── config.php  
├── public/  
│   ├── index.php  
│   ├── css/  
│   ├── js/  
│   └── uploads/  
└── models/
```

```
├── entities/
├── daos/
├── controllers/
├── views/
├── utils/
├── sql/
│   └── database.sql
├── tests/
├── .gitignore
├── README.md
└── composer.json
```

Set write permissions on `public/uploads/`

- **Windows (XAMPP):** Right-click `uploads` folder → Properties → Security → Select Users → Check “**Modify**” and “**Write**” → OK.

6. Self-Test (Core Concepts – same as before)

- MVC separation
- Singleton pattern
- `$_GET` vs `$_POST`
- `session_start()`
- Prepared statements
- `require_once` vs Composer autoloading

If unsure, review with AI or documentation.

7. Final Integration Test (XAMPP)

Step 1: Create `public/test.php` with:

```
<?php phpinfo();
```

Step 2: Start XAMPP Apache (and MySQL)

- Open XAMPP Control Panel → Start **Apache** (and **MySQL**).

Step 3: Access via browser

```
http://localhost/your-project-root/public/test.php
```

(Note: XAMPP uses `localhost` without port 8000 unless you use the built-in server.)

Step 4: Test PDO + MySQL

Create `test_db.php` **outside public** (e.g., in project root) with:

```
<?php
try {
    $pdo = new PDO('mysql:host=localhost;dbname=test_mvc;charset=utf8mb4',
    'root', '');
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "PDO connection successful!";
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
```

Run it from command line:

```
php test_db.php
```

Or place it inside `public/` and open in browser (then delete afterwards).

Checklist Summary (XAMPP)

Requirement	Status ( / )
Git installed & configured	
GitHub repo created	
XAMPP installed (PHP + MySQL)	
PHP ≥ 7.4, extensions enabled	
MySQL running via XAMPP	
Can create a database & connect	
Composer installed	
Folder structure created	
<code>public/uploads/</code> writable (Windows permissions)	
Understand core MVC & security	
PDO connection test passed	

What to do if something fails (XAMPP specific)

- **php not recognized** → Add `C:\xampp\php` to your System PATH (Environment Variables).
- **MySQL won't start** → In XAMPP Control Panel, click "Config" → "my.ini" → check port 3306 is free; also check Windows services (`Ctrl+R` → `services.msc` → look for MySQL).
- **Permission denied on uploads** → Right-click folder → Properties → Security → Edit → give `Modify` to `Users`.
- **PDO driver not found** → Open `C:\xampp\php\php.ini`, uncomment `extension=pdo_mysql`, restart Apache.

Extra: .env.example + Configuration Loader

As requested, here's a simple, secure configuration loader that works with XAMPP (and any environment). It uses a `.env` file (excluded from Git) and a `config.php` that reads it.

1. Create `.env.example` (commit this to Git)

```
# Copy this file to .env and fill in your real values
# .env is ignored by Git (add to .gitignore)

APP_NAME="PHP MVC Project"
APP_URL="http://localhost/your-project-root/public"

DB_HOST="localhost"
DB_NAME="your_database_name"
DB_USER="root"
DB_PASS=""

SESSION_NAME="myapp_session"

UPLOAD_MAX_SIZE=2097152
UPLOAD_ALLOWED_TYPES="jpg,png,pdf"
UPLOAD_DIR="C:/xampp/htdocs/your-project-root/public/uploads/"
```

2. Create `config/config.php` (loader)

```
<?php
// config/config.php
// Loads environment variables from .env file

class Config
{
```

```

private static $env = null;

private static function loadEnv()
{
    if (self::$env !== null) {
        return self::$env;
    }

    $envFile = __DIR__ . '/../.env';
    if (!file_exists($envFile)) {
        die('.env file not found. Please copy .env.example to .env and
configure it.');
```

```

    }

    $lines = file($envFile, FILE_IGNORE_NEW_LINES |
FILE_SKIP_EMPTY_LINES);
    foreach ($lines as $line) {
        if (strpos(trim($line), '#') === 0) {
            continue;
        }
        list($key, $value) = explode('=', $line, 2);
        self::$env[trim($key)] = trim($value);
    }
    return self::$env;
}

public static function get($key, $default = null)
{
    $env = self::loadEnv();
    return $env[$key] ?? $default;
}

// Convenience methods
public static function getDbConfig()
{
    return [
        'host' => self::get('DB_HOST'),
        'dbname' => self::get('DB_NAME'),
        'user' => self::get('DB_USER'),
        'password' => self::get('DB_PASS'),
    ];
}

public static function getUploadConfig()
{
    return [
        'max_size' => (int)self::get('UPLOAD_MAX_SIZE', 2097152),
        'allowed_types' => explode(',',
self::get('UPLOAD_ALLOWED_TYPES', 'jpg,png,pdf')),
        'upload_dir' => self::get('UPLOAD_DIR', __DIR__ .

```

```

'../../public/uploads/'),
    ];
}

public static function getAppUrl()
{
    return rtrim(self::get('APP_URL', 'http://localhost'), '/');
}
}

```

3. Usage example in Database.php (Singleton)

```

<?php
require_once __DIR__ . '/../config/config.php';

class Database
{
    private static $instance = null;
    private $pdo;

    private function __construct()
    {
        $dbConfig = Config::getDbConfig();
        $dsn = "mysql:host={$dbConfig['host']};dbname=
{$dbConfig['dbname']};charset=utf8mb4";
        $this->pdo = new PDO($dsn, $dbConfig['user'], $dbConfig['password'],
[
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_PERSISTENT => true
        ]);
    }

    public static function getInstance()
    {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance->pdo;
    }

    private function __clone() {}
    private function __wakeup() {}
}

```

4. Add to .gitignore


```
.env  
config/db_config.php    # if you used the old method
```

Now your credentials are **never committed** to Git, and XAMPP users simply copy `.env.example` to `.env` and adjust paths (e.g., `UPLOAD_DIR` with Windows backslashes or forward slashes).

You are now fully ready for Unit 0 – Database Schema. Let me know if you want me to generate the SQL schema (Unit 0) directly tailored to this setup.